

# Artificial Intelligence

# 人工智能实验

---

## 强化学习基础

中山大学计算机学院  
2026年春季

# 目录

## 1. 理论课内容回顾

1.1 强化学习介绍

1.2 值迭代和策略迭代

1.3 Q-learning和SARSA

1.4 深度强化学习介绍

1.5 DQN

## 2. 实验任务

2.1 值迭代和策略迭代（无需提交）

2.2 迷宫任务（无需提交）

2.3 CartPole任务

## 3. 作业提交说明

# 目录

## 1. 理论课内容回顾

1.1 强化学习介绍

1.2 值迭代和策略迭代

1.3 Q-learning和SARSA

1.4 深度强化学习介绍

1.5 DQN

## 2. 实验任务

2.1 值迭代和策略迭代（无需提交）

2.2 迷宫任务（无需提交）

2.3 CartPole任务

## 3. 作业提交说明

# 1.1 强化学习介绍

## □ 基础知识

- 强化学习（Reinforcement Learning, RL），是指一类从（与环境）交互中不断学习的问题以及解决这类问题的方法。强化学习问题可以描述为一个智能体从与环境的交互中不断学习以完成特定目标（比如取得最大奖励值）。

□ Reinforcement learning is learning what to do—how to map situations to actions—so as to maximize a numerical reward signal. ----- Richard S. Sutton and Andrew G. Barto 《Reinforcement Learning: An Introduction II》

- 形式化定义：由六元组构成的马尔可夫决策过程定义

### Markov Decision Process(MDP)

- $S$  denotes the state space
- $A$  is the action space
- $R = R(s, a)$  is the reward function
- $T: S \times A \times S \rightarrow [0,1]$  is the state transition function
- $P_0$  is the distribution of the initial state
- $\gamma$  is a discount factor
- Goal: find the optimal policy that maximizes expected reward

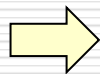


# 1.1 强化学习介绍

## □ 基础知识

- 状态价值函数(State-value Function): 用来度量给定策略的情况下, 当前状态的好坏程度

$$v_{\pi}(s) = \mathbb{E}_{\pi} \left[ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | S_t = s \right]$$



$$G_t = r_{t+1} + \gamma r_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E}[G_t | S_t = s] \\ &= \mathbb{E}_{\pi}[r_{t+1} + \gamma r_{t+2} + \cdots | S_t = s] \\ &= \mathbb{E}_{\pi}[r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \cdots) | S_t = s] \\ &= \mathbb{E}_{\pi}[r_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= \mathbb{E}_{\pi}[r_{t+1} + \gamma v(S_{t+1}) | S_t = s] \end{aligned}$$

- 动作价值函数(Action-value Function): 用来度量给定状态和策略的情况下, 采用动作的好坏程度

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} \left[ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | S_t = s, A_t = a \right]$$



$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[r_{t+1} + \gamma q(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

# 1.1 强化学习介绍

## □ 基础知识

### ■ 最优动作价值函数：

$$q^*(s, a) = R_s^a + \gamma \sum_{s' \in S} P_{ss'}^a \max_{a'} q^*(s', a')$$

□ 其中  $P_{ss'}^a$  是状态转移函数，即在状态  $s$  选择动作  $a$  转移到状态  $s'$  的概率

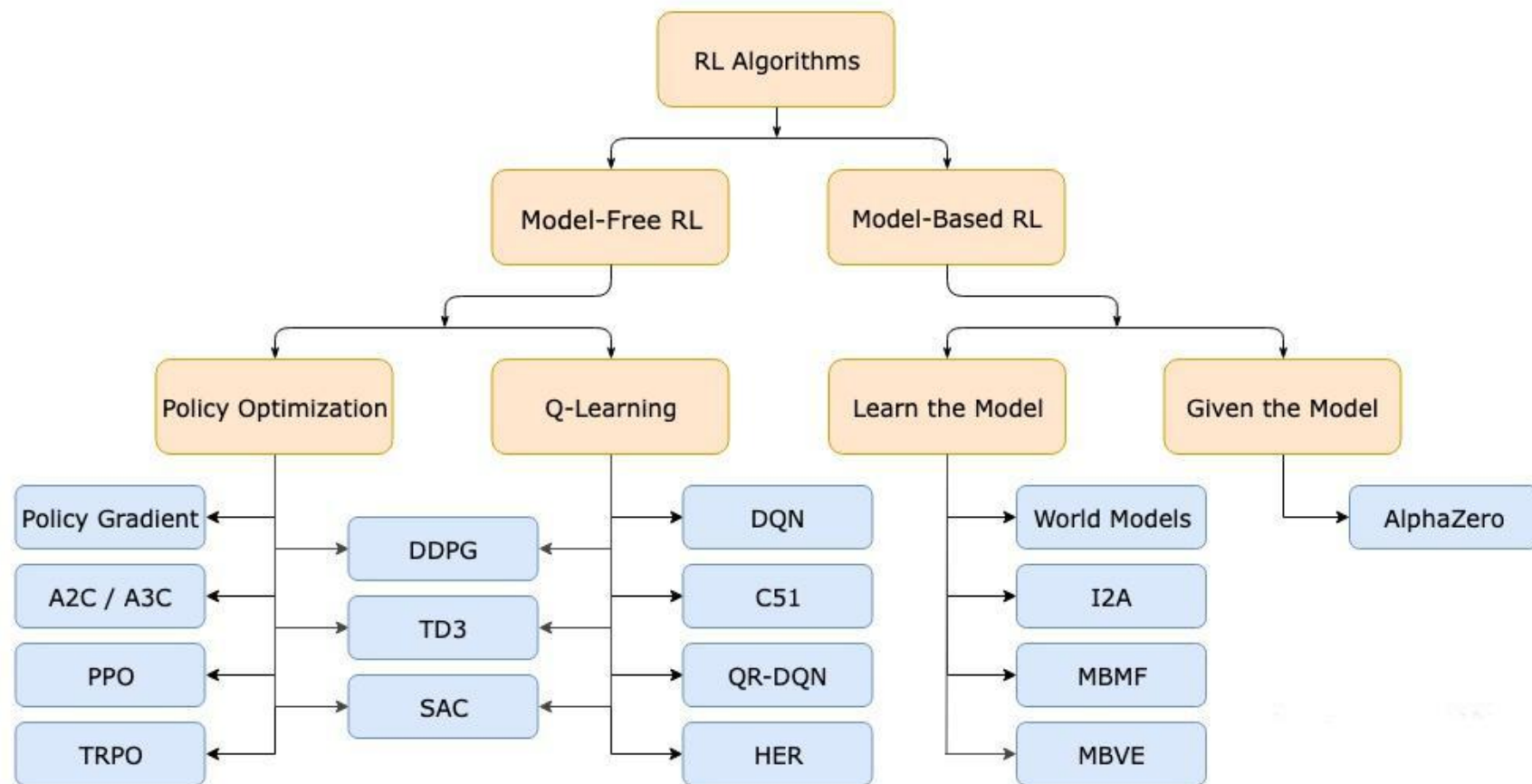
### ■ 最优策略

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \arg \max_{a \in A} q^*(s, a) \\ 0, & \text{otherwise} \end{cases}$$

# 1.1 强化学习介绍

## □ 基础知识

### ■ 强化学习算法分类

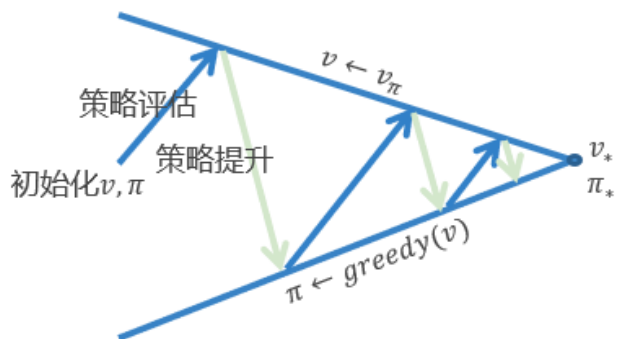


# 1.2 值迭代和策略迭代

## □ 策略迭代

### 策略迭代 (Policy Iteration)

- **策略评估** 估计原策略价值函数  $v_\pi$ 
  - ✓ 迭代策略评估
- **策略提升** 生成新策略  $\pi' \geq \pi$ 
  - ✓ 贪心策略提升



### Algorithm 1.2 策略迭代

**Input:** 策略  $\pi(s)$ , 状态转移  $P(s'|s, a)$ , 奖励函数  $R(s, a)$ , 阈值  $\epsilon$ , 衰减因子  $\gamma$ , 值函数  $V(s)$

**Output:** 最优策略  $\pi_*(s)$ , 最优值函数  $V_{\pi_*}(s)$

- 1:  $k \leftarrow 0$ ,  $\pi_k(s) \leftarrow \pi(s)$ ,  $V_k(s) \leftarrow V(s)$ 。
- 2: **for**  $k = 0, 1, 2, \dots$  **do**
- 3:   利用策略评估得到当前策略  $\pi_k(s)$  的值函数  $V_k(s)$
- 4:   利用策略改进公式 1.26 得到新策略  $\pi_{k+1}(s)$
- 5:   **if**  $\pi_{k+1}(s) = \pi_k(s)$  **then**
- 6:      $\pi_*(s) \leftarrow \pi_k(s)$ ,  $V_{\pi_*}(s) \leftarrow V_k(s)$
- 7:     **return**  $\pi_*(s)$ ,  $V_{\pi_*}(s)$
- 8:   **end if**
- 9: **end for**



# 1.2 值迭代和策略迭代

## □ 值迭代

### 值迭代 (Value Iteration)

- 迭代的调用[贝尔曼最优方程](#)进行更新
- 使用同步更新（每次迭代更新所有状态的价值）

对于  $k = 1 : H$  ( $H$  是迭代次数)

对于所有状态  $s \in \mathcal{S}$

$$v_{k+1}(s) = \max_a \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s')$$

### 不同于策略迭代，价值迭代中没有显式的策略

#### Algorithm 1.3 值迭代

**Input:** 状态转移  $P(s'|s, a)$ , 奖励函数  $R(s, a)$ , 阈值  $\epsilon$ , 衰减因子  $\gamma$ , 值函数  $V(s)$

**Output:** 最优策略  $\pi_*(s)$ , 最优值函数  $V_{\pi_*}(s)$

```
1:  $k \leftarrow 0$ ,  $V_k(s) \leftarrow V(s)$ .  
2: for  $k = 0, 1, 2, \dots$  do  
3:   for 每个状态  $s \in S$  do  
4:     利用公式 1.30 计算  $V_{k+1}(s)$   
5:   end for  
6:   if  $|V_{k+1} - V_k| < \epsilon$  then  
7:      $V_{\pi_*}(s) \leftarrow V_{k+1}$   
8:     利用公式 1.31 计算最优策略  $\pi_*(s)$   
9:     return  $\pi_*(s)$ ,  $V_{\pi_*}(s)$   
10:  end if  
11: end for
```

# 1.3 Q-learning和SARSA

## □ Q-learning算法介绍

- 是一种 **value-based** 的强化学习算法，Q即为 $Q(s,a)$ ，在某一时刻的state状态下，采取动作action能够获得收益的期望。
- 主要思想是**将state和action构建一张Q-table表存储Q值，然后根据Q值选取能够获得最大收益的动作。**
- 基于off-policy时序差分法，且使用贝尔曼方程可以对马尔科夫过程求解最优策略。
- 伪码：

1. Initialize Q-values ( $Q(s, a)$ ) arbitrarily for all state-action pairs.
2. For life or until learning is stopped...
3. Choose an action ( $a$ ) in the current world state ( $s$ ) based on current Q-value estimates ( $Q(s, \cdot)$ ).
4. Take the action ( $a$ ) and observe the outcome state ( $s'$ ) and reward ( $r$ ).
5. Update  $Q(s, a) := Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

# 1.3 Q-learning和SARSA

## □ Q-learning算法实例

- 如左图是一个迷宫游戏，小老鼠（Agent）最开始在(0,0)位置，分别在(0,1),(0,2),(1,0),(1,1),(1,2)处可获得+1,0,+2,-10,+10的奖励值。当agent位于(1,1),(1,2)时，游戏结束。Agent的动作有四个，分别是上下左右。
- 右图是初始化的Q-table，其中行表示状态，列表示动作。



|                | ← | → | ↑ | ↓ |
|----------------|---|---|---|---|
| Start          | 0 | 0 | 0 | 0 |
| Small cheese   | 0 | 0 | 0 | 0 |
| Nothing        | 0 | 0 | 0 | 0 |
| 2 small cheese | 0 | 0 | 0 | 0 |
| Death          | 0 | 0 | 0 | 0 |
| Big cheese     | 0 | 0 | 0 | 0 |

<https://blog.csdn.net/zhm2229>

# 1.3 Q-learning和SARSA

## □ Q-learning算法实例

- Q-learning根据 $\epsilon$ -greedy选择动作：以 $\epsilon$ 的概率随机选择动作，以 $1-\epsilon$ 的概率贪心的（greedy）选择动作。
- 如下图所示，从start状态随机选择向右的动作。



<https://blog.csdn.net/zhm2229>

|                | ← | → | ↑ | ↓ |
|----------------|---|---|---|---|
| Start          | 0 | 0 | 0 | 0 |
| Small cheese   | 0 | 0 | 0 | 0 |
| Nothing        | 0 | 0 | 0 | 0 |
| 2 small cheese | 0 | 0 | 0 | 0 |
| Death          | 0 | 0 | 0 | 0 |
| Big cheese     | 0 | 0 | 0 | 0 |

We move at random (for instance, right) [g.csdn.net/zhm2229](https://blog.csdn.net/zhm2229)

# 1.3 Q-learning和SARSA

## □ Q-learning算法实例

■ Q-learning使用贝尔曼方程更新：

$$\underbrace{NewQ(s, a)}_{\substack{\text{New Q value for that state and} \\ \text{that action} \\ \uparrow \\ 0.1}} = \underbrace{Q(s, a)}_{\substack{\text{Current Q} \\ \text{value} \\ \uparrow \\ 0}} + \underbrace{\alpha}_{\substack{\text{Learning Rate} \\ \uparrow \\ 0.1}} [\underbrace{R(s, a)}_{\substack{\text{Reward for taking} \\ \text{that action at that} \\ \text{state} \\ \uparrow \\ 1.0}} + \underbrace{\gamma}_{\substack{\text{Discount} \\ \text{rate} \\ \uparrow \\ 0.9}} \underbrace{\max Q'(s', a')}_{\substack{\text{Maximum expected future reward given the} \\ \text{new } s' \text{ and possible actions at that new} \\ \text{state} \\ \uparrow \\ 0}} - \underbrace{Q(s, a)}_{\substack{\text{Current Q} \\ \text{value} \\ \uparrow \\ 0}}]$$

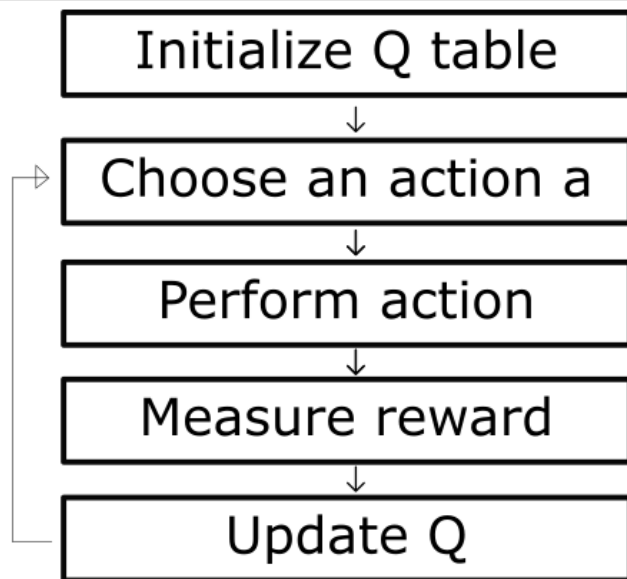
$$\gamma = 0.9, \alpha = 0.1$$

|                | ← | →   | ↑ | ↓ |
|----------------|---|-----|---|---|
| Start          | 0 | 0.1 | 0 | 0 |
| Small cheese   | 0 | 0   | 0 | 0 |
| Nothing        | 0 | 0   | 0 | 0 |
| 2 small cheese | 0 | 0   | 0 | 0 |
| Death          | 0 | 0   | 0 | 0 |
| Big cheese     | 0 | 0   | 0 | 0 |

# 1.3 Q-learning和SARSA

## □ Q-learning算法实例

- Agent在每个step的时候都会用上述方法迭代更新一次Q-table，直到Q-table不再更新，或者到达游戏设置的结束局数。



At the end of the training



# 1.3 Q-learning和SARSA

## □ SARSA算法介绍

- 和Q-learning类似，两者的区别在于：更新Q表的时候，选择的策略不同。  
Sarsa更新Q表的策略与选择动作策略一致，均采用 $\epsilon$ -greedy。而Q-learning更新Q表采用greedy策略，选择动作采用 $\epsilon$ -greedy。

```
Initialize  $Q(s, a)$  arbitrarily
Repeat (for each episode):
  Initialize  $S$ 
  Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
  Repeat (for each step of episode):
    Take action  $A$ , observe  $R, S'$ 
    Choose  $A'$  from  $S'$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
     $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$ 
     $S \leftarrow S'; A \leftarrow A';$ 
  until  $S$  is terminal
```

```
Initialize  $Q(s, a)$  arbitrarily
Repeat (for each episode):
  Initialize  $S$ 
  Repeat (for each step of episode):
    Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
    Take action  $A$ , observe  $R, S'$ 
     $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
     $S \leftarrow S';$ 
  until  $S$  is terminal
```

# 1.4 深度强化学习介绍

## □ 什么是深度强化学习？

- 传统的RL算法有个很大的问题在于它是一种表格方法，就是根据过去出现过的状态，统计和迭代Q值。这些基于表格的方法，一方面适用的状态和动作空间非常小，对于图像和高维度离散状态、连续域状态无法直接适用；另一方面对于一个状态从未出现过，这些算法是无法处理的，也就是说基于表格的算法没有对未知状态的泛化能力。
- 深度模型（Deep）的引入使得强化学习算法（RL）解决更复杂的问题
  - Deep = can process complex sensory input, 能用于处理复杂的感知输入
  - RL = can choose complex actions, 能用于选择复杂的动作
- 深度神经网络用于状态值函数、动作值函数、策略的表征



# 1.5 DQN算法

## □ DQN算法

- 将Q-learning算法和深度神经网络结合，并额外引入两个机制**经验回放**和**目标网络**
- **经验回放（Replay Buffer）**：将智能体探索环境得到的数据储存起来，然后随机采样小批次样本更新深度神经网络的参数

### □ 引入原因：

- 深度神经网络作为有监督学习模型，要求数据满足独立同分布
- Q-Learning 算法得到的样本前后是有关系的。为了打破数据之间的关联性，Experience Replay 方法通过存储-采样的方法将这个关联性打破了。

### □ 优点：

- 数据利用率高，因为一个样本被多次使用。
- 连续样本的相关性会使参数更新的方差（variance）比较大，该机制可减少这种相关性。注意这里用的是均匀随机采样

# 1.5 DQN算法

## □ DQN算法

- 将Q-learning算法和深度神经网络结合，并额外引入两个机制**经验回放**和**目标网络**
- **目标网络 (Target Network)**：额外引入一个目标网络（和Q网络具有同样的网络结构），此目标网络不更新梯度，每隔一段时间将Q网络的参数赋值给此目标网络。

### □ 引入原因：

- 深度神经网络作为有监督学习模型，要求监督数据标签是稳定的
- Q-Learning算法使用下一时刻的Q值和奖励值作为监督信号，由于每次神经网络更新后，Q值会变化，导致Q-Learning算法的监督信号不稳定。

### □ 优点：

- 一定程度降低了当前Q值和目标Q值的相关性。
- 在一段时间里目标网络的Q值是保持不变的，提高了算法稳定性。

# 1.5 DQN

## □ DQN算法

### ■ 伪码:

#### 算法 2.2 DQN 算法<sup>[66]</sup>

**Input:** 经验回放池  $B$ , 批样本大小  $B$ , 目标网络更新间隔  $d$

**Output:** 策略  $\pi$

```
1: 随机初始化 Q 值网络参数  $\phi$ , 初始化学率  $\alpha$ , 最大回合数  $N$ , 一个回合的最大时间步  $T$ , 梯度更新次数  $G$ 
2: 初始化目标值网络参数  $\phi' \leftarrow \phi$ 
3: for 每个回合  $ep = 1, 2, \dots, N$  do
4:   重置环境并获取环境的初始状态
5:   for 每个时间步  $t = 1, \dots, T$  do
6:     在时刻  $t$ , 状态为  $s$  时, 智能体根据  $\epsilon$  贪心策略和动作的值函数选择对应的动作  $a$ 
7:     根据状态转移函数, 环境转移到下一时间步状态  $s'$  并返回对应的奖励值  $r$ 
8:     将四元组  $(s, a, r, s')$  存入经验回放池  $B$  中
9:     for 每个梯度更新步  $g = 1, \dots, G$  do
10:      从经验回放池  $B$  中采样一批大小为  $B$  数据样本
11:      使用公式 (2-17) 更新神经网络参数  $\phi$ 
12:    end for  $\phi \leftarrow \phi - \alpha \frac{dQ_\phi}{d\phi}(s, a) \left( Q_\phi(s, a) - r(s, a) - \gamma \max_{a'} Q_{\phi'}(s', a') \right)$ ,
13:    if  $t$  除以  $d == 0$  then
14:      使用公式 (2-18) 更新目标网络参数  $\phi'$ 
15:    end if  $\phi' \leftarrow \phi$ , every  $d$  time steps.
16:  end for
17: end for
```

# 目录

## 1. 理论课内容回顾

1.1 强化学习介绍

1.2 值迭代和策略迭代

1.3 Q-learning和SARSA

1.4 深度强化学习介绍

1.5 DQN

## 2. 实验任务

2.1 值迭代和策略迭代（无需提交）

2.2 迷宫任务（无需提交）

2.3 CartPole任务

## 3. 作业提交说明

# 2.1 值迭代和策略迭代（无需提交）

## □ 值迭代和策略迭代

- 在Frozen Lake环境中实现值迭代和策略迭代算法。
- 要求：
  - 在给定的代码框架下补充代码。
  - 最终智能体学习的策略能完成任务。

## 2.2 迷宫任务（无需提交）

### □ 迷宫任务（无需提交）

- 在给定迷宫环境中实现Q-learning和Sarsa算法。
- 要求：
  - 在给定的代码框架下补充代码。
  - 最终智能体学习的策略能完成迷宫任务。

## 2.3 CartPole任务

### □ CartPole任务

- 在CartPole环境中实现DQN算法。
- 要求：
  - 在给定的代码框架下补充代码。
  - 最终的reward至少收敛至180.0（注意：这里的reward指一局游戏的奖励值之和）。
  - 评分标准：以算法收敛的reward大小、收敛所需的样本数量给分。reward越高（最大是200）、收敛所需样本数量越少，分数越高。

# 目录

## 1. 理论课内容回顾

1.1 强化学习介绍

1.2 值迭代和策略迭代

1.3 Q-learning和SARSA

1.4 深度强化学习介绍

1.5 DQN

## 2. 实验任务

2.1 值迭代和策略迭代（无需提交）

2.2 迷宫任务（无需提交）

2.3 CartPole任务

## 3. 作业提交说明



### 3. 作业提交说明

- ❑ 压缩包命名为：“学号\_姓名\_作业编号”，例：  
20260611\_张三\_实验六。
- ❑ 每次作业文件下包含两部分：code文件夹和实验报告PDF文件。
  - code文件夹：存放实验代码；
  - PDF文件格式参考发的模板。
- ❑ 如果需要更新提交的版本，则在后面加\_v2，\_v3。如第一版是“学号\_姓名\_作业编号.zip”，第二版是“学号\_姓名\_作业编号\_v2.zip”，依此类推。
- ❑ 截至日期：**2026年6月25日晚24点。**
- ❑ 提交邮箱：[zhangyc8@mail2.sysu.edu.cn](mailto:zhangyc8@mail2.sysu.edu.cn)。